ThermaLoop AI
Five Integrated Strategies for Energy-Efficient AI Inference

A Technical Research Report on Model Routing, DVFS, Prefill/Decode Disaggregation, Thermal-Aware Scheduling, and Cooling Diagnostics

Research prototype | October 2026 | Technical explanatory manuscript

Scope and evidence statement. This report explains the five GPU-side optimization strategies in the ThermaLoop AI browser-based digital twin. The existing 64-GPU simulation is educational and uses synthetic workloads and reduced-order power/thermal models; it has not been validated against measurements on a deployed 64-GPU installation. Values and worked examples labeled “illustrative” are not experimental findings.

Abstract

Energy-efficient large language model (LLM) inference requires more than reducing the number of generated tokens. Model selection, accelerator clock policy, separation of inference phases, workload placement, and hardware-health monitoring affect how much electrical energy is required to serve a request while meeting quality and latency constraints. This report describes five cooperating strategies implemented or approximated in ThermaLoop AI: (1) intelligent model routing, (2) dynamic voltage and frequency scaling (DVFS), (3) prefill/decode disaggregation, (4) thermal-aware job scheduling, and (5) predictive cooling diagnostics. It provides the control logic, mathematical objectives, illustrative calculations, integration architecture, expected trade-offs, and a validation protocol. Each section distinguishes methods established in published work from simplified mechanisms represented in the current prototype. The central performance metric is energy per accepted response or energy per successfully served token, accompanied by service-level and quality measurements. No claimed percentage energy saving is treated as validated until matched experiments are performed.

Keywords: AI inference; GPU data center; LLM routing; DVFS; disaggregation; thermal scheduling; anomaly detection; energy per token.

1. Research problem and system boundaries

1.1 Why token count alone is not the objective

A text generation workload has input tokens (prompt and context), output tokens (the generated response), model-specific computational work, and a service-level objective (SLO). A strategy may serve exactly the same input and output tokens but consume less GPU energy by using smaller models, maintaining more favorable clock settings, or avoiding throttling. Conversely, reducing the output token count does not necessarily imply an equal reduction in electricity usage because prompt prefill, idle GPU power, queueing, CPU and networking overhead, and cooling continue to consume power.

Energy intensity = Total site energy (J) / Successfully served tokens

Energy per accepted response = Total site energy (J) / Accepted responses

The denominator must be well-defined. “Successfully served” means that a request completed under specified deadlines and quality criteria. For cross-model comparisons, accepted responses is usually a more defensible normalization than raw output tokens, since different models can generate different response lengths. Report both indicators and absolute system energy, not a single percentage.

1.2 System modeled in the prototype

The current browser twin represents 64 separate synthetic GPU devices arranged in 8 racks (8 GPUs per rack). Each GPU has an individual synthetic speed factor, thermal factor, utilization, clock multiplier, power estimate, temperature state, task counter, and token counter. The simulation creates stochastic jobs each minute, places them into a bounded queue, executes work against per-GPU minute capacities, and updates electrical/thermal state. The heat recovery network may be visualized elsewhere in the project, but this manuscript concentrates exclusively on operation of the AI computing system.

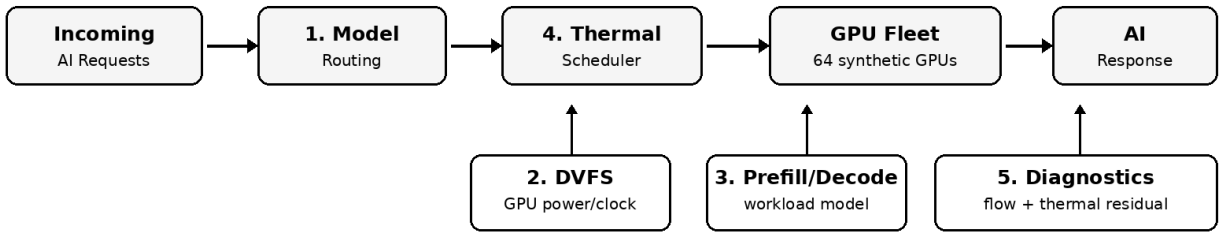


Figure 1. Conceptual request-control architecture and the five optimization functions (schematic, not a physical wiring diagram).

Model element	Prototype assumption	Scientific interpretation
GPU fleet	64 synthetic devices; 8 racks	Logical model; not an inventory of measured GPUs
GPU idle/dynamic power	0.14 kW idle plus up to 0.64 kW dynamic component per modeled GPU	Illustrative constants; not a measured hardware specification
Non-GPU IT power	19 kW	Synthetic infrastructure baseline
Time step	1 simulated minute	Aggregate workload approximation, not kernel timing
Queue capacity	7,000 requests	Bounded buffer with modeled drops
Power or quality data	Reduced-order synthetic estimates	Requires telemetry and task-level benchmarks for validation

Table 1. Selected parameters observed in the current ThermaLoop AI prototype.

1.3 End-to-end request life cycle

1. A request enters with a prompt, intended output, priority, and deadline.
2. The router determines whether a smaller model is sufficient or whether a stronger model is necessary.

3. The queue and scheduler assign the resulting compute work to an eligible GPU according to capacity and, where enabled, thermal state.
4. Prefill processes the input context; decode repeatedly generates output tokens. Disaggregation changes the allocation of these phases in a real serving architecture.
5. DVFS or a power governor adjusts GPU clock/power behavior to match operating load and SLO requirements.
6. Telemetry and diagnostic logic look for a cooling fault or a mismatch between predicted and observed temperature.
7. The system returns an answer, records completed work, elapsed time, quality, and consumed energy, and updates the next control decision.

2. Strategy I — Intelligent model routing

2.1 Function and rationale

Different questions require different model capability. A basic factual lookup or straightforward classification might be handled adequately by a smaller language model, whereas complex reasoning, difficult technical inference, or high-risk tasks may require a more capable model. Intelligent model routing seeks the least costly model that is sufficiently likely to meet the required answer quality. It does not simply truncate tokens; it changes the amount of computation used per input token and output token. The underlying trade-off is investigated in RouteLLM [1].

2.2 Inputs, decision, and control outputs

Input	Meaning	Example observable
Prompt features	Length, topic, estimated difficulty and format	Token count, classifier score
Candidate model set	Available models and supported contexts	Small model, large model
Quality threshold	Minimum predicted acceptability	Pass rate or evaluator score
SLO / deadline	Maximum acceptable response delay	P95 latency, TTFT and TPOT
Cost estimate	Compute time, GPU-seconds and energy	Predicted J/request
Routing decision	Model identity passed to serving scheduler	Model A vs. Model B

Table 2. Inputs and outputs of a model router.

The ideal decision is constrained optimization rather than a fixed difficulty threshold. For a request x and model m, let Q(m,x) be predicted response quality, E(m,x) predicted energy, and L(m,x) predicted latency. Choose the model with minimum E among those meeting the required quality Q_min and latency L_max. An application may add monetary cost and fairness penalties.

$$m^* = \arg\min_m E(m, x) \quad \text{subject to } Q(m, x) \geq Q_{min} \text{ and } L(m, x) \leq L_{max}$$

2.3 Illustrative worked example

Assume 100 independent requests: 60 simple requests and 40 difficult requests. A larger model costs 12 kJ/request; a smaller model costs 4 kJ/request for the target response length. If all requests use the large model, energy is 100 × 12 = 1,200 kJ. If the router safely sends 60 simple requests to the small model and 40 difficult requests to the large model, the model-execution portion is 60 × 4 + 40 × 12 = 720 kJ. This is a 40% reduction in the modeled execution energy for this deliberately simplified example. It does not include router overhead, extra retries, non-GPU IT load, cooling energy, or any loss in answer quality.

Routing example: (1,200 - 720) / 1,200 × 100% = 40% (illustrative)

2.4 What ThermaLoop AI actually implements

In the current source, each synthetic request receives a random complexity score. When routing is enabled, the request is assigned to the small model if complexity <= 0.56. Small and large routes have different synthetic GPU-work multipliers in both the prefill and decode stages. The simulation also creates a probability-based proxy for answer acceptability. This is not a trained router, not an actual inference call, and not a measured model-quality benchmark. There are no production model weights or genuine evaluation labels in the browser simulation.

2.5 Failure cases and measurements

- Misrouting: a difficult task incorrectly sent to the small model may fail a quality check, leading to retry cost or unacceptable answers.
- Router overhead: classification itself adds CPU/GPU work and latency.
- Model residency: keeping multiple model weights in VRAM may increase memory pressure and cause evictions or reload delays.
- Quality drift: the difficulty classifier may become unreliable with a new domain, language, or evolving model population.

Evaluate with a labeled request set and a blind response evaluator. Report routing accuracy, accepted-response rate, router overhead, energy per accepted response, retries, TTFT, TPOT, and P95 end-to-end latency. Always compare against all-large, all-small, and a quality-matched policy.

3. Strategy II — Dynamic voltage and frequency scaling (DVFS)

3.1 Function and electrical rationale

GPU frequency governs how rapidly computation may proceed. For many CMOS components, dynamic power is approximately proportional to effective switched capacitance, switching activity, the square of supply voltage, and operating frequency. Voltage and frequency are coupled by the hardware operating curve, which makes the power/performance relationship nonlinear. Lower clock and voltage may reduce power, but they can also lengthen execution and increase queueing. Thus the relevant objective is workload energy and deadline satisfaction, not minimum instantaneous wattage alone.

$$P_{dynamic} \approx \alpha \cdot C_{eff} \cdot V^2 \cdot f$$

$$E_{job} = \int P_{job}(t)\, dt \approx P_{average} \times execution_time$$

Commercial GPU control is constrained by driver, firmware, device capability, thermal headroom, and vendor-approved clock or power-limit interfaces. A high-level digital twin should not assume arbitrary voltage adjustment on every GPU; a “DVFS” slider in the prototype is an abstract clock/power governor, not evidence that actual silicon voltage was changed.

3.2 Controller policy

An ideal governor uses queue depth, priority, predicted time to completion, thermal headroom, and clock transition cost. At low load, it can attempt an energy-efficient state; at high load or near a deadline, it may need higher frequency. If a workload is memory-bandwidth bound, increasing compute clock may produce little speedup but still increase power. The control loop must therefore consider measured throughput-per-watt and achieved response latency.

3.3 Illustrative break-even calculation

Suppose a GPU job takes 20 seconds at 280 W: energy = 5,600 J. A lower-performance setting uses 190 W but extends runtime to 26 seconds: energy = 4,940 J. The energy reduction is (5,600 - 4,940)/5,600 = 11.8%; runtime increases by 30%. This change is favorable only if the longer runtime does not violate request deadlines or create additional queueing and idle-energy penalties. Both power and runtime values here are hypothetical.

3.4 What the current prototype computes

The synthetic policy calculates a shared clock factor using queue work, with a floor of 0.72 and a ceiling of 1.0, and combines it with each GPU’s previous utilization. The per-GPU active power model is:

$$P_{GPU,i}\ (kW) = 0.14 + 0.64 \cdot u_i \cdot c_i^{2.15} \cdot h_i$$

Here u_i is synthetic utilization, c_i is a synthetic clock multiplier, and h_i is a device-specific thermal factor. Capacity scales with the clock multiplier. The exponent 2.15 is an engineering simplification used by the demo, not a universal GPU law or a measured voltage/frequency curve. The prototype does not explicitly integrate measured voltage, hardware clock-change latency, kernel-level stalls, or actual accelerator power caps.

3.5 Validation and conditions where it fails

- Measure GPU board power, SM clock, workload completion times, temperature, and request latency under several allowed clock/power limits.
- Identify throughput-per-watt and joules-per-accepted-request rather than selecting the smallest wattage.
- Test both interactive bursty traffic and large offline batches; the optimal operating point can differ.
- Check deadline misses, power throttling, and whether the workload is compute-limited or memory-limited.

4. Strategy III — Prefill/decode disaggregation

4.1 How autoregressive inference works

An autoregressive LLM normally has two service phases. Prefill reads the prompt and computes intermediate states, including the key/value (KV) cache, for the initial context. Decode produces subsequent

tokens iteratively while reusing that cache. Prefill commonly benefits from parallel compute on many prompt tokens; decoding may depend more strongly on memory bandwidth and per-step latency. Combining both phases on the same GPUs can create interference and complicate resource planning. DistServe [2] studies assigning the phases to different GPU groups.

4.2 Architecture of a physically disaggregated implementation

A true disaggregated system requires a prefill worker pool, a decode worker pool, a KV-cache transfer mechanism, request admission, and inter-pool scheduling. The prefill worker sends the context state to a decoder over an interconnect. Transfer time depends on cache size, bandwidth, serialization and synchronization; if the transfer overhead exceeds the benefit from phase specialization, performance can deteriorate.

$$T_{disagg} \approx T_{prefill} + T_{transfer(KV\ cache)} + T_{decode} + T_{queue}$$

$$KV\ transfer\ time \approx KV\ bytes / effective\ bandwidth + transfer\ overhead$$

For a transformer-like model, a coarse KV-memory estimate scales with the number of layers, sequence tokens, KV heads, head dimension, and bytes per value, multiplied by 2 for keys and values. Quantization and architectural details change the actual footprint. A proper design checks memory capacity and network bandwidth before allowing a request into the decode pool.

$$KV_{bytes} \approx 2 \cdot L \cdot N_{tokens} \cdot H_{KV} \cdot d_{head} \cdot bytes_{per_element}$$

4.3 Worked example and trade-off

Illustrative timing: a colocated request spends 140 ms on prefill and 600 ms on decode/interference, totaling 740 ms. A specialized configuration might require 115 ms prefill, 550 ms decode, and 35 ms transfer, totaling 700 ms (5.4% faster). But if the KV transfer rises to 120 ms, total time becomes 785 ms, or 6.1% slower than the colocated version. These figures are hypothetical and demonstrate why bandwidth and network placement are key experimental variables.

4.4 What the ThermaLoop AI browser model does and does not do

The current simulation divides synthetic job work into prefill and decode components. With the “disagg” switch enabled, it reduces prefill work by a factor of 1.13 and decode work by a factor of 1.10, while adding 0.34 synthetic GPU-seconds of per-request communication overhead. Both components are aggregated into a single job-work value before being scheduled on the same 64-device pool. It does not instantiate dedicated prefill and decode GPU pools, does not transfer a real KV cache, and does not measure TTFT/TPOT separately. Thus the webpage demonstrates an assumed trade-off, not a functioning physical disaggregated serving stack.

4.5 What should be measured

- Time to first token (TTFT), time per output token (TPOT), request goodput at simultaneous TTFT and TPOT SLO thresholds.
- KV-cache bytes transferred per request, NIC utilization, network transfer latency, VRAM allocation and evictions.
- GPU utilization, watts and joules per accepted response for both colocated and disaggregated serving.

- Resource imbalance between prefill workers and decode workers, plus admission failures during bursts.

5. Strategy IV — Thermal-aware workload scheduling

5.1 Purpose

Different GPUs can have different inlet-coolant conditions, heatsink effectiveness, baseline temperatures and accumulated work. A naive placement policy can repeatedly select a hot device, increasing the chance of thermal throttling. A thermal-aware scheduler considers current load and temperature when assigning jobs so that safe processing capacity remains available and deadlines are met. In Thermaloop AI this strategy additionally defers some flexible work when modeled heat demand is low and the thermal buffer is already relatively full. That heat-demand coupling is a research hypothesis rather than a proven energy-saving policy.

5.2 Scheduling variables

Variable	Definition	Why it matters
u_i	Current utilization of GPU i	Avoids overloaded devices
T_i	Measured or estimated GPU temperature	Tracks thermal headroom
W_j	Remaining work of job j	Determines whether it fits a GPU timeslot
d_j	Job deadline	Prevents excessive deferral
M_i, M_j	Free VRAM vs. request VRAM requirement	Required for a real scheduler; not hard-enforced in this browser allocation
L_j	Expected latency / SLO	Required for deployment-grade guarantees
R_i	GPU health / cooling availability	Avoids unavailable or degraded hardware

Table 3. Minimum inputs for a constraint-aware scheduling policy.

A useful placement score can combine normalized utilization, temperature above a threshold, projected finish time, and migration cost. It should first reject GPUs that lack the required VRAM or cannot meet the deadline. Only eligible devices should compete on the score; otherwise the scheduler can give an attractive but physically impossible assignment.

$$\text{score}(i,j) = w_u \cdot u_i + w_T \cdot \max(0, T_i - T_{safe}) + w_L \cdot \text{projected_delay}(i,j)$$

5.3 Current scheduling implementation

When enabled, the prototype sorts queued jobs primarily by deadline, searches all 64 GPUs for sufficient remaining minute capacity, and scores candidate placement using normalized load plus a penalty of 0.023 per degree above 55 °C, with small speed/scan-order adjustments. If no GPU can finish an entire job within the remaining simulated minute, it may allocate a partial timeslot to the device with greatest remaining capacity. Jobs still unfinished at the next minute may be reassigned with zero modeled migration cost. The current per-request allocator does not enforce VRAM capacity or modeled network-transfer penalties at this

granular level. Separately, a scale/economics calculator includes approximate VRAM and latency checks, but that does not make them enforced scheduler constraints.

5.4 Two contrasting examples

Example A (benefit): GPU 1 is at 82 °C with 80% use, while GPU 2 is at 60 °C with 30% use and enough VRAM. If both can complete an incoming task before its deadline, assigning the task to GPU 2 can reduce the risk of throttling on GPU 1. Example B (failure): GPU 2 has only 4 GiB free VRAM and the request requires 12 GiB. A real scheduler must reject GPU 2 even if it is colder; temperature cannot overrule memory capacity.

5.5 Evaluation

- Measure P50/P95/P99 response latency, deadline miss rate, queue length, completed requests, output tokens, per-GPU temperature maxima, throttle events and site energy.
- Compare against simple round-robin, least-loaded scheduling, and earliest-deadline-first using the same request arrival trace.
- Test failure of one GPU or rack, asymmetric thermal conditions, long-context jobs, batch deadlines, and memory fragmentation.
- Inspect workload fairness and unintended starvation, particularly when jobs are repeatedly deferred for energy reasons.

6. Strategy V — Predictive cooling diagnostics and safe response

6.1 Why diagnostics matter for AI compute

A partially blocked cooling path, reduced liquid flow, failed fan or sensor anomaly can cause GPU temperature to rise despite normal computational demand. Without observation, the system may continue placing work on affected accelerators until thermal limits cause throttling or protective shutdown. A diagnostic subsystem can combine observed temperature, expected temperature, coolant flow and device-health state to identify suspicious operating conditions. “Predictive” ideally means identifying deterioration before a failure; in the current prototype the mechanism is better described as threshold-based anomaly detection.

6.2 Residual-based anomaly detection

A physics- or telemetry-based model predicts GPU temperature from power and cooling conditions. A residual is the difference between observed and predicted temperature. Sustained positive residuals across nearby devices, together with unusually low measured coolant flow, can indicate a cooling problem. But the residual alone does not prove that flow restriction is the cause: sensor calibration error, hot inlet air, workload transients, or incorrect baseline models can produce similar signatures.

$$\mathbf{Residual_i(t) = T_{observed,i}(t) - T_{predicted,i}(t)}$$

$$\mathbf{Raise\ alert\ if\ flow < flow_limit\ AND\ \max_i Residual_i > residual_limit}$$

6.3 How the current twin simulates a fault

The prototype represents normal primary-loop coolant flow as 4.0 kg/s. Its injected cooling fault changes the synthetic system flow to 2.05 kg/s and adds a synthetic temperature rise of 17 °C to GPUs in racks 6 and 7 (using human-readable 1-based rack numbers); other racks receive a smaller rise. When diagnostics is enabled, the simplified rule flags “flow restriction” if flow falls below 3 kg/s and maximum residual exceeds 4.5 °C. An associated protection multiplier derates selected GPU clocks to 0.73 of their previous setting after the fault event. These thresholds and injection magnitudes are simulation assumptions, not certified hardware safety settings.

6.4 Safe operational architecture for a real deployment

8. Observe: collect temperature, power, clock, throttle flags, inlet/outlet coolant temperature, flow, pump status and sensor health.
9. Detect: filter noisy measurements, calculate residuals and alert severity, confirm persistence over multiple samples.
10. Diagnose: cross-check flow, pump pressure, physical rack topology and neighboring devices before assigning likely cause.
11. Protect: follow manufacturer- and facility-approved controls for power caps, draining jobs, failover, maintenance and cooling fallback.
12. Recover: require evidence that thermal headroom has returned before restoring normal request admission.

6.5 Testing false alarms and fault detection

Inject benign power bursts, hot ambient conditions, stale sensors and a simulated flow restriction. Report true positive rate, false alarm rate, detection delay, time above thermal thresholds, dropped requests, requests completed, and energy for both diagnostics-enabled and diagnostics-disabled systems. A useful detector is one that catches faults without causing excessive unnecessary derating. Decreased power during a fault must not be mistaken for a positive efficiency gain if fewer legitimate requests are completed.

7. How the five strategies work together

7.1 Control layers and interactions

Model routing acts at the application/model-choice layer. Prefill/decode disaggregation acts at the inference-serving architecture layer. Thermal-aware scheduling selects devices and determines timing. DVFS changes per-device operating state. Cooling diagnostics observes failures and can constrain the other controllers. These controls interact: routing changes the distribution of model weights and the GPU-seconds per request; disaggregation changes where work and KV-cache traffic appear; scheduling redistributes load and temperature; DVFS changes execution speed and power; diagnostics can remove capacity by initiating derating.

Strategy	Primary objective	Potential adverse effect	Required guardrail
Model routing	Reduce compute for suitable requests	Quality loss, model-switch overhead	Quality threshold + retries counted

Strategy	Primary objective	Potential adverse effect	Required guardrail
DVFS	Reduce joules under safe utilization	Deadline misses or queue growth	Latency-aware clock floor
Disaggregation	Improve phase-specific goodput	KV transfer overhead, idle pools	Bandwidth, TTFT/TPOT SLO
Thermal-aware scheduling	Avoid hot spots and congestion	Starvation, VRAM infeasibility	Hard resource checks + fairness
Diagnostics	Detect cooling anomalies, protect GPUs	False alarms, lost throughput	Multi-sensor confirmation + recovery logic

Table 4. Optimization objectives and minimum safeguards.

7.2 A practical decision sequence

13. Read global state: request arrivals, per-device power and temperature, free VRAM, queue depth, health and network capacity.
14. Classify request difficulty and route to an eligible model, accounting for expected quality and route overhead.
15. Estimate prefill work, KV-cache footprint and decode work; choose colocated or disaggregated phase execution only when transfer constraints permit.
16. Select a GPU or worker pool with sufficient VRAM and thermal health, while meeting the request deadline.
17. Apply a permissible clock/power policy based on workload and latency slack.
18. During execution, monitor telemetry and reduce admissions or migrate future work if a verified cooling issue arises.
19. Record site joules, tokens, accepted responses, latency distributions, and quality results; update control policies.

Joint tuning is required. The savings attributable to each technique are not generally additive. For example, routing can leave GPUs lightly loaded, making DVFS more effective, but disaggregation may add network energy. Remove-one-strategy ablation experiments show incremental contributions conditional on other controls; their percentages must not be summed as if they were independent.

8. Mathematical metrics and worked system comparison

8.1 Power and energy accounting

$$P_{site} = \Sigma_i P_{GPU,i} + P_{otherIT} + P_{cooling} + P_{pumps} + P_{network} + P_{aux}$$

$$E_{site}\ (kWh) = \Sigma_t P_{site}(t) \times \Delta t_{hours}$$

$$\text{Energy savings } (\%) = 100 \times [E_{before} - E_{after}] / E_{before}$$

This accounting must use energy over the same equivalent workload, not simply instantaneous site power. If a faster system completes more work within the same 60 minutes, the wattage might be similar while joules per accepted request improve significantly. The reverse can also happen: lower watts but a growing queue or declining accepted-response rate is not adequate evidence of a useful optimization.

8.2 Service and quality metrics

Goodput = Requests satisfying quality AND latency SLO / elapsed time

SLO compliance (%) = 100 × Requests meeting SLO / Arrived requests

Energy per accepted request (J) = 3.6 × 10^6 × E_site(kWh) / N_accepted

When output lengths vary, separately report joules per accepted input+output token, joules per generated output token, and joules per accepted request. Report TTFT and TPOT for streaming inference. The denominator should be defined consistently across Before and After.

8.3 Fully labeled hypothetical comparison

Metric	Baseline	Optimized	Interpretation
Requests arriving in 60 min	10,000	10,000	Exact same trace
Accepted before deadline	8,000	9,000	Better goodput
Site energy	100 kWh	96 kWh	4% absolute energy decrease
Accepted output tokens	2,000,000	2,250,000	More useful work completed
J per accepted request	45,000 J	38,400 J	14.7% lower
J per accepted output token	180 J	153.6 J	14.7% lower

Table 5. Invented example solely to demonstrate correct normalization; not output from a ThermaLoop AI experiment.

In Table 5, energy per accepted request is 100 × 3.6 million / 8,000 = 45,000 J before and 96 × 3.6 million / 9,000 = 38,400 J after. The resulting reduction is 14.7%, although total site energy drops by just 4%. This discrepancy is legitimate because accepted work changes, but it requires transparency: the optimized policy serves more work, not fewer tokens.

9. Research protocol and reproducibility

9.1 Baselines and ablations

Run a matched baseline with all five features off and a full-feature configuration with all five features on. Repeat a five-way ablation in which one feature at a time is removed from the full configuration. Additionally run single-feature configurations with only that feature enabled. Use the same arrivals, input/output token lengths, model-quality targets, fault timing, environmental conditions and random seeds across paired trials. Publish the configuration and event trace.

Experiment	Changed variable	Principal measured outcomes
E0: all off	Baseline policy	Energy, accepted requests, P95, peak GPU temperature
E1: routing only	Enable routing	Quality, retries, joules/accepted request
E2: DVFS only	Enable clock policy	GPU clocks, watts, deadline compliance
E3: phase split only	Enable disaggregation	TTFT, TPOT, KV traffic and goodput

Experiment	Changed variable	Principal measured outcomes
E4: thermal scheduling only	Enable thermal placement	Hot spots, backlog and deadline misses
E5: diagnostics only	Inject fault; enable detection	Detection delay, false positives and availability
E6: five on	All controls active	Joint outcome; interactions
E7: leave-one-out	Disable one of five from E6	Non-additive conditional contributions

Table 6. Proposed reproducible experiment matrix.

9.2 Test scenarios

At minimum include steady low demand, heavy saturated demand, a sudden traffic burst, long-context requests, mixed interactive/batch workload, cooling restriction, and device failure. For each scenario run multiple seeds and report mean, standard deviation or bootstrap confidence intervals. Distinguish actual measured workloads from synthetic traces. A true GPU-based validation should record power and temperature using supported device telemetry (for example, NVIDIA DCGM metrics [3]), and a proper LLM server should record per-request TTFT/TPOT and quality decisions.

9.3 Validation targets

- Power prediction: compare simulated power against measured GPU board power and full-site meter data; report MAE and bias.
- Thermal prediction: compare observed vs. predicted temperature over low/high load and fault scenarios; report MAE in °C and residual distributions.
- Serving model: compare predicted accepted requests, queue length, TTFT, TPOT and token throughput against measurements.
- Control fairness: assess deadline misses and tail latency separately for interactive and flexible batch tasks.
- Generalization: fit parameters on one subset of runs; evaluate on held-out ambient temperatures, request mixes and GPU operating states.

10. Limitations, novelty and conclusions

10.1 The distinction between a research prototype and a measured deployment

The current browser implementation demonstrates a reproducible educational design space. It supports individual synthetic GPUs, controlled toggles, scenarios, Before/After comparisons, per-minute histories and data export. However, the model router is threshold-based rather than learned; DVFS is a clock/power proxy rather than measured silicon voltage control; disaggregation is modeled as phase-specific work multipliers plus overhead rather than physical GPU-pool separation; memory and latency rules are not enforced throughout every per-GPU placement decision; and cooling diagnosis uses injected synthetic fault conditions and threshold tests. The report intentionally does not assert that any current simulated saving is real-world verified.

10.2 Research contribution that can be evaluated

The potential contribution of ThermaLoop AI is a transparent coupled system-level methodology: evaluate multiple inference control strategies together under common quality, latency, workload, GPU thermal-state and infrastructure-energy constraints. These five optimization ideas are not individually novel claims. The scientific question is whether coordinated control produces a reproducible improvement in accepted inference work per unit of facility energy while meeting operational constraints. A credible contribution requires an implementation trace, defined baselines, calibrated model parameters, and independent experiments.

10.3 Conclusion

The five strategies operate at different levels of a GPU data center and should be treated as a coordinated control system rather than five independent “token-saving” switches. Model routing reduces unnecessary computation on easier requests; DVFS targets the energy-performance operating point; prefill/decode disaggregation reduces service-phase interference if network and memory costs are acceptable; thermal-aware scheduling seeks reliable use of heterogeneous GPUs; and cooling diagnostics protects service continuity under abnormal conditions. Together they motivate a measurable objective: minimize energy per accepted response while maintaining answer quality, latency and reliability. The next research step is systematic validation using real telemetry and serving measurements, not the presentation of larger unverified savings percentages.

References

[1] I. Ong et al., “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv:2406.18665, 2024 (revised 2025). https://arxiv.org/abs/2406.18665

[2] Y. Zhong et al., “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving,” arXiv:2401.09670, 2024. https://arxiv.org/abs/2401.09670

[3] NVIDIA, “DCGM Exporter Metrics,” official DCGM documentation, accessed October 2026. https://docs.nvidia.com/datacenter/dcgm/latest/reference/dcgm-exporter-metrics.html

[4] NVIDIA, “Power and Performance,” IGX Developer Guide, accessed October 2026. https://docs.nvidia.com/igx/user-guide/latest/SW/power-perf.html

[5] ThermaLoop AI, “GPU Research Twin / Scale & Economics Browser Prototype,” local source code reviewed for this report (2026). Note: internal prototype, not a peer-reviewed or externally validated dataset.